LEAPPs FIELD GUIDE

Extracting, Processing, and Querying Apple Unified Logs from an iOS Device

The updated workflow: acquire the logs, preserve the archive, process with iLEAPP, and work the results in LAVA.

Alexis Brignoni

Updated July 29, 2026

Apple Unified Logs can show locks, app launches, connectivity, navigation, and more. Here is the updated workflow for acquiring them, processing them with iLEAPP, and working the results in LAVA.

Here is a data source I do not think examiners can afford to ignore anymore: Apple Unified Logs.

Want to know when an iPhone locked or unlocked? Which app launched? When Wi-Fi, Bluetooth, Airplane Mode, or the flashlight changed state? Whether navigation started, the volume buttons were pressed, the phone changed orientation, or someone took a screenshot? The Unified Logs may know.

Not always. Not forever. But often enough, and with enough detail, that this evidence belongs in the normal iOS examination workflow.

I first wrote about this workflow in May 2025. At the time, LAVA was still coming, the dedicated iLEAPP artifacts were future work, and the native Apple tooling had fewer options. A lot has changed.

Short version: acquire the complete logs, preserve the original archive, use Apple's own tooling to export a JSON working copy, let iLEAPP turn that monster file into SQLite and dedicated artifacts, and do the analysis in LAVA.

Long version: keep reading.

Download the printable PDF edition.

One warning before we start: Unified Log messages change across devices and operating-system versions. A pattern that works on one iPhone is not automatically universal. Validate the findings that matter and correlate them with the rest of the case.

What Apple Unified Logs are

Apple describes the unified logging system as a central store for telemetry from across the operating system. Instead of writing everything into ordinary text files, Apple keeps the data in memory and on disk in its own format. Console, the `log` command, and the OSLog framework are the native ways in.

On an iOS full-file-system extraction, the two paths we care about are:

```
/private/var/db/diagnostics
/private/var/db/uuidtext
```

The `diagnostics` directory holds the `.tracev3` data. The `uuidtext` directory holds support data needed to resolve parts of it. You need both. A `.logarchive` is simply the bundle that brings those pieces together in a form Apple's tools understand.

Here is the catch: these logs are enormous and they do not live forever. Retention depends on time-to-live behavior, log level, storage pressure, and other conditions. There is no safe promise that the evidence will remain for a fixed number of days. Acquire it early.

Acquisition options

Collect from a connected device with macOS

If you have the device and a Mac, this is the easiest route. Connect and trust the iPhone or iPad, open Terminal, and run:

```
sudo log collect --device --output "/path/to/case-device.logarchive"
```

If more than one device is connected, recent versions of `log` may also give you `--device-name` and `--device-udid`. Run `log help collect` on the acquisition Mac and save the output with your notes. Apple changes this command. We will come back to that.

Do not get clever and limit the acquisition to a narrow time range just because the archive is big. Collect everything available first. Hash it. Preserve it. Narrow it later.

If the native command is not your route, there are other good options:

- UFADE, Christian Peter's open-source Apple-device acquisition tool.

- iOS Unified Logs Acquisition, Lionel Notari's macOS acquisition tool, which records device and case information, log statistics, and archive hashes.
- Commercial forensic tools that explicitly preserve the Unified Log components.

Reconstruct a working `.logarchive` from a full-file-system extraction

What if you already have a full-file-system extraction instead of a collected `.logarchive`? Build one from a working copy:

1. Create a new directory with a `.logarchive` extension.
2. Copy the complete contents of `diagnostics` and `uuidtext` into its root. Copy the contents, not the two parent directories themselves.
3. Add a compatible `Info.plist` at the root of the package.
4. Test the derivative archive with `log stats` or Console on the analysis Mac.

This used to be the easy part. Create a tiny `Info.plist`, add `OSArchiveVersion`, and move on.

Then macOS 26.4 changed the rules.

Johann Polewczyk did the work to figure out exactly what changed and documented it in The Info.plist File in an Apple Unified Logs .logarchive Package. The old one-key plist is no longer enough on current macOS.

Select the correct `OSArchiveVersion`

First, the version number has to match the operating-system generation represented by the evidence:

macOS evidence	iOS/iPadOS evidence	`OSArchiveVersion`
10.12-10.12.5	10.0-10.2	2
10.12.6-10.13.6	10.3-11.x	3
10.14-11.x	12.x-14.x	4
12.x-26.x	15.x-26.x	5

One important distinction: this table is for selecting the value manually. Johann's current `logarchive_info.py` does not detect the evidence OS and choose among versions 2 through 5. The script is built for contemporary archives and explicitly writes `OSArchiveVersion` 5. If you are reconstructing an older archive, use the appropriate historical value and validate it with an analysis system that supports that generation.

For iOS or iPadOS 15 through 26, that means version 5:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
"http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
<key>OSArchiveVersion</key>
<integer>5</integer>
</dict>
</plist>
```

That was enough on older analysis systems. Starting with macOS 26.4, `log show` also expects archive-specific timing and stream metadata.

Johann's current script can generate metadata for four log streams when matching `.tracev3` files are present: `SpecialMetadata`, `SignpostMetadata`, `PersistMetadata`, and `HighVolumeMetadata`.

Each stream's `OldestTimeRef` contains `ContinuousTime`, `UUID`, and `WallTime` values derived from that archive. The script also builds `LiveMetadata` and `EndTimeRef` from the most recent `.timesync` boot UUID and the latest matching catalog continuous time.

If `version.plist` is present, the script can also copy its `Identifier` into `SourceIdentifier` and place available `ttl01`, `ttl03`, `ttl07`, `ttl14`, and `ttl30` values under `SpecialMetadata` → `TTL`.

None of these are generic values. The tempting shortcut would be to copy a working plist from another archive.

Do not do that. Those UUIDs and time references belong to the source evidence.

The good news is that Johann also released logarchive_info.py. It reads the target archive's `.tracev3`, `.timesync`, and `version.plist` files and builds the right `Info.plist` at the archive root. If you are working on macOS 26.4 or later, use it:

```
python3 logarchive_info.py "/path/to/case-device.logarchive"
```

Record the script version or commit you used and hash the generated plist with the rest of the working package.

Remember what this package is: a derivative you created for analysis. Keep the untouched source directories, document how you built it, hash the result, and confirm Apple's tools can read it. When possible, compare the timestamps and record counts against another validated rendering.

If you want this workflow as a picture, Tim Korver has a visual acquisition-and-preservation flow in his Apple Unified Log repository. His Thesis Friday site and CLI cheatsheet belong in your bookmarks too.

Inspect the archive before converting it

Before turning a large archive into a much larger JSON file, get the statistics:

```
log stats --archive "/path/to/case-device.logarchive" \
--style human --overview
```

Save that output. Later, when you are staring at a database with millions of rows, you will want to know whether the time span and counts line up with the archive you started with.

Apple has also been quietly improving the `log` command:

- `--end` support in `log stats`, allowing statistics for the same bounded period used during conversion.
- `log repack`, which can create a smaller derivative archive containing records that match a predicate.
- Additional collection filtering and selection options.

These options depend on the version of macOS doing the analysis. Record `sw_vers`, save `log help`, and keep the exact commands you ran. If you repack or filter an archive, keep the complete original. Always.

Convert the archive to JSON for iLEAPP

This is the bridge from macOS to the rest of the workflow. Use Apple's own command to render the archive into JSON:

```
log show --archive "/path/to/case-device.logarchive" \
--style json --info --debug > "/path/to/logarchive.json"
```

The `--info` and `--debug` flags matter. Leave them out and `log show` normally gives you only the default level. That means missing records before the analysis even starts.

Also, make room. A multi-gigabyte `.logarchive` can become a JSON file tens of gigabytes in size. In my original test, a 1.63 GB archive became 29.19 GB of JSON. That is not a typo.

Hash the JSON when it finishes and leave the source `.logarchive` alone.

iLEAPP accepts names matching `logarchive*.json`, so `logarchive-device01.json` is fine. Keep only the intended export in the input directory. You do not want iLEAPP guessing between two 30 GB files.

If you already know the time of interest, `--start` and `--end` can make conversion much faster. Just be honest about what you created: it is a filtered derivative. Document the time zone and bounds, keep the full archive, and compare the result against `log stats` using the same window.

Process the JSON with iLEAPP

Now hand the giant JSON file to iLEAPP:

1. Download the current iLEAPP release.
2. Select the directory containing the `logarchive*.json` file as the input.
3. Select an output directory.
4. In Available Modules, select the Unified Logs modules. The raw logarchive module must run before the artifacts that depend on it.
5. Start processing.

The raw module streams the file instead of trying to load tens of gigabytes into memory. It writes the fields we care about into `_lava_artifacts.db`:

- Timestamp, normalized to UTC
- Row number
- Process image path
- Process ID
- Subsystem
- Category
- Event message
- Trace ID

That database is the working set. The `.logarchive` and JSON still hold fields iLEAPP does not import, so keep them.

And no, iLEAPP does not try to cram eighteen million rows into an HTML report. Good. That would be useless.

The full table is a LAVA-only artifact. Open the completed project in LAVA to filter, tag, and export the records that matter. If you want to write SQL directly, DB Browser for SQLite works too.

Unified Log artifacts currently supported by iLEAPP

When I published the original article, dedicated Unified Log artifacts were something we planned to build.

The future arrived.

I went back to the query itself, because counting only the report names badly understates what is supported.

As of July 29, 2026, the local `logarchive.py` query contains 134 `LIKE` clauses representing 132 unique message predicates. Those predicates cover 23 evidentiary themes:

Evidentiary theme	What the query looks for	Unique predicates
Screenshots	Screenshot capture messages	1
System time changes	Clock-shift messages	1
Walking activity	Potential walking bouts identified by BoutDetector	1
Contact details	Messages indicating the presence of a contact name and phone number	1
Charging	Charger connection-state changes	1
Motion state	Motion-state transitions	1
CarPlay	CarPlay connection events	1

Evidentiary theme	What the query looks for	Unique predicates
Accessories	Accessory connection and accessory-information changes	2
Siri speech requests	The start of speech-request sessions	1
Device orientation	Received and effective orientation messages	2
Authentication	Match start, face, authentication, Apple Account authentication, and related state transitions	5
Screen, lock, and biometric state	Screen state, lock/unlock, device lock status, and biometric match messages	9
Touch and process visibility	SpringBoard icon touches and process-visibility changes	2
Wi-Fi	Power state, association, reachability, SSIDs, forgotten or removed networks, scans, joins, and known-network activity	24
Driving mode	Vehicular state, Driving Focus/Do Not Disturb While Driving, and related mode events	7
Airplane Mode	Enabled, disabled, active, inactive, and state-change variants	11
Bluetooth, calls, and wireless audio links	Bluetooth power and state, device connections, hands-free activity, call state, voice audio, A2DP streaming, and link quality	28
Audio playback and volume	Playback state, volume-button presses, volume changes, and playback queue invalidation	7
Physical controls, brightness, and ringer	Volume control, Emergency SOS button gestures, brightness changes, and ringer/silent state	6
Executed applications	Icon taps, application launches, and transitions	3
Flashlight	Flashlight controller and AVFlashlight activity	2
Personal Hotspot	Tethering and wireless-modem state changes	3
Navigation	Route start, maneuver guidance, distance prompts, arrival, and destination messages	13

That is the real supported-artifact surface of the broad logarchive artifacts query. It is a LAVA-only collection, so those themes do not all appear as separate HTML report names.

The same script currently registers 13 iLEAPP outputs: the raw import, the broad artifact query, and 11 dedicated standard reports:

iLEAPP artifact	What it surfaces	Output
logarchive	The complete set of JSON records imported into SQLite	LAVA only
logarchive artifacts	All 23 themes and 132 unique message predicates summarized above	LAVA only

iLEAPP artifact	What it surfaces	Output
logarchive time change	Messages indicating that the system clock shifted	Standard
logarchive flashlight	Flashlight on/off and controller activity	Standard
logarchive executed apps	Application launch, icon-tap, and transition activity	Standard
logarchive personal hotspot	Personal Hotspot and tethering state changes	Standard
logarchive airplane mode	Airplane Mode state changes	Standard
logarchive lock status	Screen lock/unlock and biometric match activity	Standard
logarchive WiFi status	Wi-Fi state, association, scans, joins, known networks, and removal activity	Standard
logarchive Bluetooth status	Bluetooth power, connection, disconnection, hands-free, A2DP, and related state activity	Standard
logarchive audio status	Audio playback, volume-button, and volume-change activity	Standard
logarchive motion state transitions	Motion-state transition messages	Standard
logarchive navigation	Route start, turn guidance, arrival, and destination-related prompts	Standard

Read this part carefully: "supported" means iLEAPP knows how to look for the message patterns we have researched. It does not mean every iOS version emits the same message, and an empty artifact does not prove an event never happened.

If the dedicated artifact is empty, go wider. Check logarchive artifacts, then search the raw logarchive table. That is also where the next artifact is waiting to be found.

The live list is always available in the iLEAPP source module and the LEAPPs artifact browser.

Querying the database

The dedicated artifacts get you to the obvious hits fast. The raw table is where the fun begins.

Once you find something interesting, pivot into the full logarchive around that timestamp, process, subsystem, or message. Pull the records before and after it. One log line tells you an event happened. The surrounding lines often tell you why.

If you are using a SQLite viewer, confirm the actual table and column names first. Framework output can change.

A basic time-window query looks like:

```
SELECT
timestamp,
row_number,
process_image_path,
process_id,
subsystem,
category,
event_message,
trace_id
FROM logarchive
WHERE timestamp BETWEEN :start_utc AND :end_utc
ORDER BY timestamp, row_number;
```

To search by message and process context:

```
SELECT *
FROM logarchive
WHERE lower(process_image_path) LIKE '%springboard%'
AND lower(event_message) LIKE '%volume%'
ORDER BY timestamp, row_number;
```

Things I keep in mind while querying:

- Work in UTC unless the case requires a documented local-time presentation.
- Use the row number as a stable secondary sort key when many events share a timestamp.
- Pull a window before and after the event of interest; surrounding messages often explain the action.
- Correlate Unified Logs with Biome, KnowledgeC, application databases, power logs, location sources, and file-system timestamps.
- Treat private or redacted fields, missing log levels, message loss, clock changes, and TTL expiration as limitations.
- Validate high-value patterns on the relevant iOS version and hardware whenever possible.

Research and tools

No one person owns this research, and the target keeps moving. These are the resources I keep close:

- Apple Logging documentation - Apple's overview of the unified logging system.
- Apple OSLog documentation - programmatic access to historical log data.
- iLEAPP - conversion of the Apple JSON export into LAVA/SQLite output and dedicated Unified Log artifacts.
- LAVA - the LEAPPs viewer for large and standard artifact outputs.
- Lionel Notari's iOS Unified Logs - extensive artifact research, articles, references, and "Unified Logs of the Week."
- Lionel Notari's acquisition tool - guided collection with reporting, statistics, and hashes.
- Lionel Notari's parsing-tool article - a macOS parser that builds complete and filtered databases, supports custom rules, and performs conversion quality checks.
- Lionel Notari on major `log` command updates - `log repack`, filtering changes, and bounded `log stats`.
- Thesis Friday by Tim Korver - ongoing Apple Unified Log artifact research across iOS, macOS, watchOS, CarPlay, physical interactions, and more.
- Tim Korver's Apple Unified Log repository - acquisition/preservation process flow and CLI material.
- Tim Korver's Apple Unified Log CLI cheatsheet - collection, display, predicates, statistics, event types, and example queries.
- Johann Polewczyk's .logarchive Info.plist research - OSArchiveVersion mapping, the macOS 26.4 metadata requirements, and the reconstruction methodology.
- Johann Polewczyk's logarchive_info.py - generates the evidence-specific `Info.plist` required by current versions of macOS.
- UFADE - open-source Apple-device acquisition by Christian Peter.
- DB Browser for SQLite - a general-purpose SQLite query tool.

Before you call it done

- Acquire early and preserve the complete Unified Log data.
- Hash the original `.logarchive`, the exported JSON, and important derivatives.
- Record the acquisition Mac, analysis Mac, OS versions, tool versions, commands, time zones, and filters.
- Use the correct `OSArchiveVersion`; on macOS 26.4 and later, generate the additional evidence-specific plist metadata.
- Export with `--info --debug` when the goal is a comprehensive JSON working copy.
- Compare database counts and time bounds against native `log stats` where possible.
- Use iLEAPP's dedicated artifacts for fast triage and the full table for context and novel research.
- Validate important patterns; do not interpret an empty parser result as proof of absence.

Apple Unified Logs are not an edge-case data source anymore. Acquire them. Preserve them. Query them.

And when you find a pattern nobody has documented yet, write it down and send it back to the community. That is how this list grows.

For questions, updates, and my current contact links, visit abrignoni.github.io.